

A Developer’s Guide: Modeling and Implementing Intelligent Systems with COH and GISMOL

Table of Contents

- 1. [Introduction](#)
- 2. [The COH 9-Tuple – A Quick Refresher](#)
- 3. [Setting Up GISMOL](#)
- 4. [Your First COH System: Smart Thermostat](#)
- 5. [Adding Neural Components \(N\)](#)
- 6. [Adding Constraints \(I, T, G\) and Reasoners](#)
- 7. [Adding Daemons \(D\) for Real-Time Monitoring](#)
- 8. [Managing Hierarchies and Repositories \(C, E\)](#)
- 9. [Natural Language Integration \(NLP Module\)](#)
- 10. [Testing and Simulation](#)
- 11. [Best Practices and Common Pitfalls](#)
- 12. [Conclusion](#)

1. Introduction

Constrained Object Hierarchies (COH) is a neuroscience-grounded, mathematical framework for building intelligent systems. It unifies hierarchy, learning, and constraints into a single formalism. **GISMOL** (General Intelligent System Modelling Language) is a Python implementation of COH, providing ready-to-use components for rapid development.

This tutorial will teach you how to:

- **Model** any intelligent system using the 9-tuple.
- **Implement** it with GISMOL’s `core`, `neural`, `nlp`, and `reasoners` modules.
- **Run, test, and extend** your system.

We assume you have basic Python knowledge (classes, dictionaries, NumPy). No deep learning or formal logic background is required, though helpful.

2. The COH 9-Tuple – A Quick Refresher

Every intelligent system in COH is defined by nine components:

Symbol	Name	What it does	Example
C	Components	Hierarchical parts (sub-objects)	<code>TemperatureSensor</code> , <code>Heater</code>
A	Attributes	State variables	<code>current_temp</code> , <code>heater_on</code>
M	Methods	Executable actions	<code>measure_temp()</code> , <code>set_heater()</code>
N	Neural	Learnable, adaptive models	<code>TempPredictor</code> (neural net)

Symbol	Name	What it does	Example
E	Embedding	Unified semantic representation	3-D vector of (temp, target, hour)
I	Identity	Invariant constraints (must always hold)	current_temp ≥ 5°C (freeze protection)
T	Triggers	Event–Condition–Action rules	IF temp < target-0.5 → heater_on
G	Goals	Optimisation objectives	Minimise temp-target + 0.01*energy
D	Daemons	Continuous monitors (run in background)	OvershootMonitor

Key insight: Intelligence emerges from the interplay of these nine facets. You don't need all of them for every simple system, but the framework is designed to be **functionally non-redundant** – each component adds essential capability.

3. Setting Up GISMOL

GISMOL is a pure Python package. Clone or copy the modules as described in the implementation appendix.

3.1. Installation (from source)

```
git clone https://github.com/hongxueharriswang/gismol_t_toolkit.git
cd gismol
pip install -e .
```

Or simply place the `gismol/` folder next to your script.

3.2. Quick verification

```
from gismol import COHObject, COHRepository
print("GISMOL ready")
```

4. Your First COH System: Smart Thermostat

We'll build a **Smart Thermostat** step by step. The full code is available in the previous example; here we explain each piece.

4.1. Create the COHObject

```
from gismol.core import COHObject

thermostat = COHObject(name="LivingRoomThermostat")
```

4.2. Add Attributes (A)

```
thermostat.add_attribute("current_temp", 20.0) # °C
thermostat.add_attribute("target_temp", 21.0) # °C
thermostat.add_attribute("heater_on", 0) # 0/1
thermostat.add_attribute("energy_used", 0.0) # kWh
thermostat.add_attribute("hour", 0) # 0-23
```

4.3. Add Methods (M)

Methods are ordinary Python functions that take `self` (the COHObject) and other arguments. Use `execute_method()` to call them with constraint checking.

```
def measure_temp(self):
    # Simulate temperature change based on heater and outside temp
    current = self.get_attribute("current_temp")
    heater = self.get_attribute("heater_on")
    # ... compute new_temp
    self.add_attribute("current_temp", new_temp)
    return new_temp

thermostat.add_method("measure_temp", measure_temp)
```

Important: Inside a method, always use `self.get_attribute()` and `self.add_attribute()` to read/write state. This ensures constraints are re-checked automatically.

4.4. Add Identity Constraints (I)

Identity constraints are invariants – they must be `True` for all reachable states.

```
thermostat.add_identity_constraint({
    'name': 'freeze_protection',
    'specification': 'current_temp >= 5',
    'severity': 10, # 1-10, 10 most severe
    'category': 'safety'
})
```

The `specification` is a string that will be evaluated by a **reasoner**. GISMOL includes reasoners for numeric comparisons, logical operators, and domain-specific rules.

4.5. Add Trigger Constraints (T)

Triggers are **Event-Condition-Action** rules. They are evaluated when relevant events occur (like a method call or time step).

```
thermostat.add_trigger_constraint({
    'name': 'heat_on_rule',
    'specification': 'WHEN current_temp < target_temp - 0.5 DO
set_heater(1)',
    'priority': 'HIGH'
})
```

The specification language is intentionally simple; you can write **WHEN condition DO action**. The action must be a method name.

4.6. Add Goal Constraints (G)

Goals define what the system should optimise. They are used by planners or reinforcement learning components.

```
thermostat.add_goal_constraint({
    'name': 'comfort_energy_balance',
    'specification': 'MINIMIZE (current_temp - target_temp)^2 + 0.01 *
energy_used'
})
```

4.7. Run the System

```
thermostat.initialize_system() # validates all constraints
thermostat.start_daemons()   # launches background monitors

# Simulate a loop
for hour in range(24):
    thermostat.add_attribute("hour", hour)
    thermostat.execute_method("measure_temp")
    # triggers automatically fire when conditions become true
    time.sleep(0.1)
```

5. Adding Neural Components (N)

Neural components are **learnable, parameterised functions**. They are subclasses of **NeuralComponent** and integrate seamlessly with constraints.

5.1. Create a Simple Predictor

```

from gismol.neural import Regressor

class TempPredictor(Regressor):
    def forward(self, x):
        # x is a numpy array of features
        return super().forward(x) # simple linear/MLP

predictor = TempPredictor(name="temp_predictor", input_dim=4, output_dim=1)
thermostat.add_neural_component("predictor", predictor)

```

5.2. Use the Predictor in a Method

```

def anticipate_cold(self):
    feat = np.array([self.get_attribute("current_temp"),
                     self.get_attribute("target_temp"),
                     self.get_attribute("hour")/24,
                     self.get_attribute("occupancy")])
    pred = self.get_neural_component("predictor").forward(feat)
    if pred < self.get_attribute("target_temp") - 1:
        self.execute_method("set_heater", 1)

```

5.3. Training

Neural components have a `train_component(dataset, epochs)` method. You can also use the optimizers from `gismol.neural.optimizers` (e.g., `ConstraintAwareOptimizer`) to respect constraints during training.

```

dataset = [...] # list of (input_features, target_temperature)
predictor.train_component(dataset, epochs=10)

```

6. Adding Constraints (I, T, G) and Reasoners

GISMOL includes a powerful reasoner subsystem. Each constraint is evaluated by a **reasoner** appropriate for its category.

6.1. Built-in Reasoners

- **BaseReasoner**: numeric comparisons, dot-notation.
- **GeometricReasoner**: distances, positions.
- **TemporalReasoner**: time-based constraints (`within`, `before`).
- **CausalReasoner**, **ProbabilisticReasoner**, **SafetyReasoner**, etc.

6.2. Registering Reasoners

```
from gismol.reasoners import TemporalReasoner, SafetyReasoner

thermostat.constraint_system.register_reasoner("temporal",
TemporalReasoner())
thermostat.constraint_system.register_reasoner("safety", SafetyReasoner())
```

Now when you add a constraint with `category: 'temporal'`, it will use the temporal reasoner.

6.3. Custom Reasoners

Subclass `BaseReasoner` and implement `evaluate()`:

```
class MyCustomReasoner(BaseReasoner, reasoner_type="custom"):
    def evaluate(self, constraint, context):
        # parse constraint.specification and return bool
        return True
```

Then register it.

7. Adding Daemons (D) for Real-Time Monitoring

Daemons are **background threads** that continuously monitor the system. They can enforce constraints that cannot be captured by triggers alone (e.g., trending conditions).

7.1. Built-in Daemons

- `IdentityConstraintDaemon`: re-checks identity constraints periodically.
- `TriggerConstraintDaemon`: watches for event conditions.
- `GoalConstraintDaemon`: tracks progress toward goals.

7.2. Custom Daemon

```
from gismol.core.daemons import ConstraintDaemon

class OvershootMonitor(ConstraintDaemon):
    def __init__(self, parent, interval=60):
        super().__init__(parent, interval)
        self.violation_start = None

    def check(self):
        current = self.parent.get_attribute("current_temp")
        target = self.parent.get_attribute("target_temp")
        if current > target + 2.0:
            if self.violation_start is None:
                self.violation_start = time.time()
            elif time.time() - self.violation_start > 600:
```

```
        print("Overshoot >2°C for 10 minutes!")
    else:
        self.violation_start = None
```

Add it to your object:

```
thermostat.daemons['overshoot'] = OvershootMonitor(thermostat, interval=60)
thermostat.start_daemons()
```

Daemons run in the background until `stop_daemons()` is called.

8. Managing Hierarchies and Repositories (C, E)

8.1. Hierarchical Components (C)

COHObjects can have children, forming a DAG (directed acyclic graph). Use `add_child()` and `remove_child()`.

```
sensor = COHObject("TempSensor")
heater = COHObject("Heater")
thermostat.add_child(sensor)
thermostat.add_child(heater)
```

When you call `thermostat.initialize_system()`, all children are initialised recursively.

8.2. Embedding (E) for Semantic Similarity

Embeddings are vector representations of objects. They enable semantic operations like similarity search.

```
from gismol.neural.embeddings import EmbeddingModel

class ThermostatEmbedding(EmbeddingModel):
    def embed(self, obj):
        return np.array([obj.get_attribute("current_temp")/40,
                        obj.get_attribute("target_temp")/40,
                        obj.get_attribute("hour")/24])

thermostat.add_neural_component("embedder", ThermostatEmbedding(),
is_embedding_model=True)
```

Now you can compute semantic distance:

```
dist = thermostat.semantic_distance(other_thermostat)
```

8.3. COHRepository – Managing Collections

COHRepository stores multiple COHObjects and their relationships. Useful for multi-agent systems.

```
repo = COHRepository()
repo.add_object(thermostat)
repo.add_object(another_thermostat)
repo.add_relation(thermostat, another_thermostat, "neighbor",
"building_wing")
repo.set_focus_object("LivingRoomThermostat")
matches = repo.find_semantic_matches("high temperature")
```

9. Natural Language Integration (NLP Module)

The **gismol.nlp** module lets you build constraint-aware conversational agents.

9.1. Simple Dialogue Manager

```
from gismol.nlp import COHDialogueManager

dialogue = COHDialogueManager(repository=repo)
response = dialogue.respond("What is the current temperature?")
```

The dialogue manager automatically consults the focus object and respects constraints (e.g., does not reveal sensitive data if a constraint prohibits it).

9.2. Intent Recognition and Response Generation

```
from gismol.nlp import IntentRecognizer, ConstraintAwareResponseGenerator

recognizer = IntentRecognizer()
intent = recognizer.recognize_intent("set temperature to 22 degrees")
# intent = {'intent': 'set_temp', 'parameters': ('22',)}

generator = ConstraintAwareResponseGenerator()
response = generator.generate_response(intent['intent'],
intent['parameters'], [thermostat])
```

9.3. Parsing Constraints from Text

```
from gismol.nlp import ConstraintParser

parser = ConstraintParser()
parsed = parser.parse("temperature must be less than 30 degrees")
```



```
constraints = parser.to_coh_constraints(parsed)
for c in constraints:
    thermostat.add_identity_constraint(c)
```

10. Testing and Simulation

10.1. Unit Testing Constraints

```
def test_constraint():
    context = thermostat.get_context()
    constraint = thermostat.identity_constraints[0]
    assert thermostat.constraint_system.validate_single(constraint,
context)
```

10.2. Simulation Loops

The thermostat example showed a simple time-step simulation. For more complex environments, create a separate `World` class and pass it to your `COHObject` as an attribute.

10.3. Logging and Monitoring

GISMOL uses Python's `logging` module. Enable debug logs to see daemon activity:

```
import logging
logging.basicConfig(level=logging.DEBUG)
```

11. Best Practices and Common Pitfalls

✓ Do's

- **Always** call `initialize_system()` after building your object – it validates all constraints.
- **Use** `execute_method()` instead of calling methods directly – this enforces triggers and identity constraints.
- **Keep constraints simple** – complex logic belongs in methods or reasoners.
- **Prefer triggers for reactive rules** (e.g., “if distance < 1, brake”) and daemons for continuous monitoring (e.g., “if average error > 0.5 over 10 minutes”).
- **Use embeddings** for semantic operations – they are more efficient than hand-coded distances.
- **Leverage the repository** when you have many interacting objects.

✗ Don'ts

- **Don't create cycles** in the component DAG – GISMOL will raise `HierarchyCycleError`.
- **Don't put heavy computation inside constraint specifications** – they are evaluated frequently.
- **Don't ignore daemon intervals** – setting them too low can kill performance.

- **Don't modify attributes directly** inside methods without using `add_attribute` – you'll bypass constraint checks.

Debugging Tips

- Use `thermostat.constraint_system.validate_all(context)` to see which constraints fail.
- Set a breakpoint inside a daemon's `check()` method.
- Print the `context` dictionary to see what values are being passed to reasoners.

12. Conclusion

You now have the tools to model and implement intelligent systems using COH and GISMOL. The 9-tuple provides a complete, structured way to capture hierarchy, learning, constraints, and real-time monitoring – all in a single, coherent framework.

Next steps:

- Experiment with the five complete examples (thermostat, dialogue bot, vacuum cleaner, trading agent, adaptive cruise control).
- Extend them with your own neural components, constraints, and daemons.
- Read the theoretical paper to understand the mathematical guarantees (soundness, completeness, functional non-redundancy).

We welcome contributions, bug reports, and success stories. Happy coding!

For more details, refer to the GISMOL API documentation and the COH theory paper.